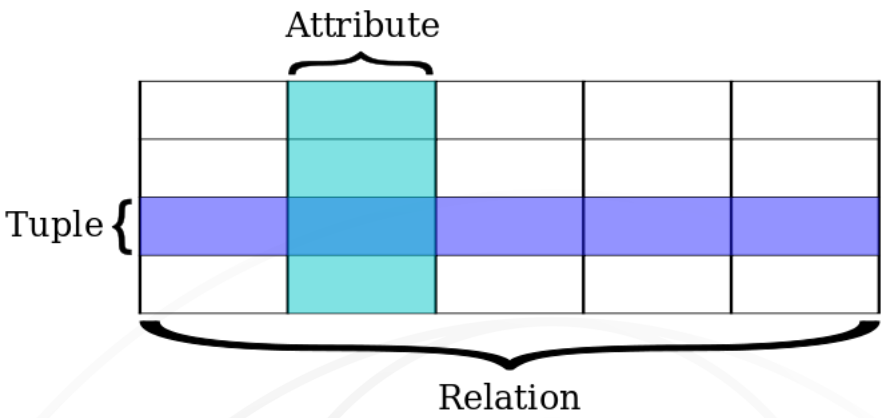
Databases, Part 2

FINTECH 512
Software Engineering

Duke

Duke

Relational Model



- Predominate storage model
- Defined by E.F. Codd at IBM in 1970

- Organizes data into tables of rows and columns

Row	Tuple / Record	Data set representing a single item
Column	Attribute / field	Labeled element of a record
Table	Relation	Set of records with the same attributes

Duke

Relational Model

Customer

cust_id	fname	lname
1	George	Blake
2	Sue	Smith

Product

product_cd	name
CHK	Checking
SAV	Savings
MM	Money market
LOC	Line of credit

Account

account_id	product_cd	cust_id	balance
103	CHK	1	$75.00
104	SAV	1	$250.00
105	CHK	2	$783.64
106	MM	2	$500.00
107	LOC	2	0

Transaction

txn_id	txn_type_cd	account_id	amount	date
978	DBT	103	$100.00	2004-01-22
979	CDT	103	$25.00	2004-02-05
980	DBT	104	$250.00	2004-03-09
981	DBT	105	$1000.00	2004-03-25
982	CDT	105	$138.50	2004-04-02
983	CDT	105	$77.86	2004-04-04
984	DBT	106	$500.00	2004-03-27

Duke

From 'Learning SQL', Beaulieu, https://learning.oreilly.com/library/view/learning-sql-3rd/9781492057604/

Relational Model: Doesn't this seem like a pain? Why not just have one table?

- Good relational design avoids redundancy… makes it easier to manage large amounts of data
  - What if someone moves? Changes their name?
- The relational model provides
  - Guidance on how to avoid redundancy and the problems it brings
  - Separation of physical and logical organization of data
  - Integrity rules
  - Query language(s) to combine and summarize data in powerful ways

Single table…

Name	Account	Transaction	Amount	Date	Balance
George Blake	Checking	Debit	$100.00	2004-01-22	$100.00
George Blake	Checking	Credit	$25.00	2004-02-05	$75.00
George Blake	Savings	Debit	$250.00	2004-03-09	$250.00
Sue Smith	Checking	Debit	$1000.00	2004-03-25	$1000.00
Sue Smith	Checking	Credit	$138.50	2004-04-02	$861.50
Sue Smith	Checking	Credit	$77.86	2004-04-04	$783.64
Sue Smith	Money Market	Debit	$500.00	2004-03-27	$500.00

Duke

Single table…

Name	Account	Transaction	Amount	Date	Balance
George Blake	Checking	Debit	$100.00	2004-01-22	$100.00
George Blake	Checking	Credit	$25.00	2004-02-05	$75.00
George Blake	Savings	Debit	$250.00	2004-03-09	$250.00
Sue Smith	Checking	Debit	$1000.00	2004-03-25	$1000.00
Sue Smith	Checking	Credit	$138.50	2004-04-02	$861.50
Sue Smith	Checking	Credit	$77.86	2004-04-04	$783.64
Sue Smith	Money Market	Debit	$500.00	2004-03-27	$500.00

1. If George Blake's Savings debit were deleted, we would lose the fact that he has a Savings account ('Deletion anomaly')
2. Sue Smith's Line of Credit Account is Missing ('Insert anomaly')
3. If she changed her name, every transaction would need to be updated ('Update anomaly')

Duke

Relational Model Terminology

Table (Relation)

Column (Attribute)

Primary key

Row (Tuple)

Account

account_id	*product_cd*	*cust_id*	*balance*
103	CHK	1	$75.00
104	SAV	1	$250.00
105	CHK	2	$783.64
106	MM	2	$500.00
107	LOC	2	0

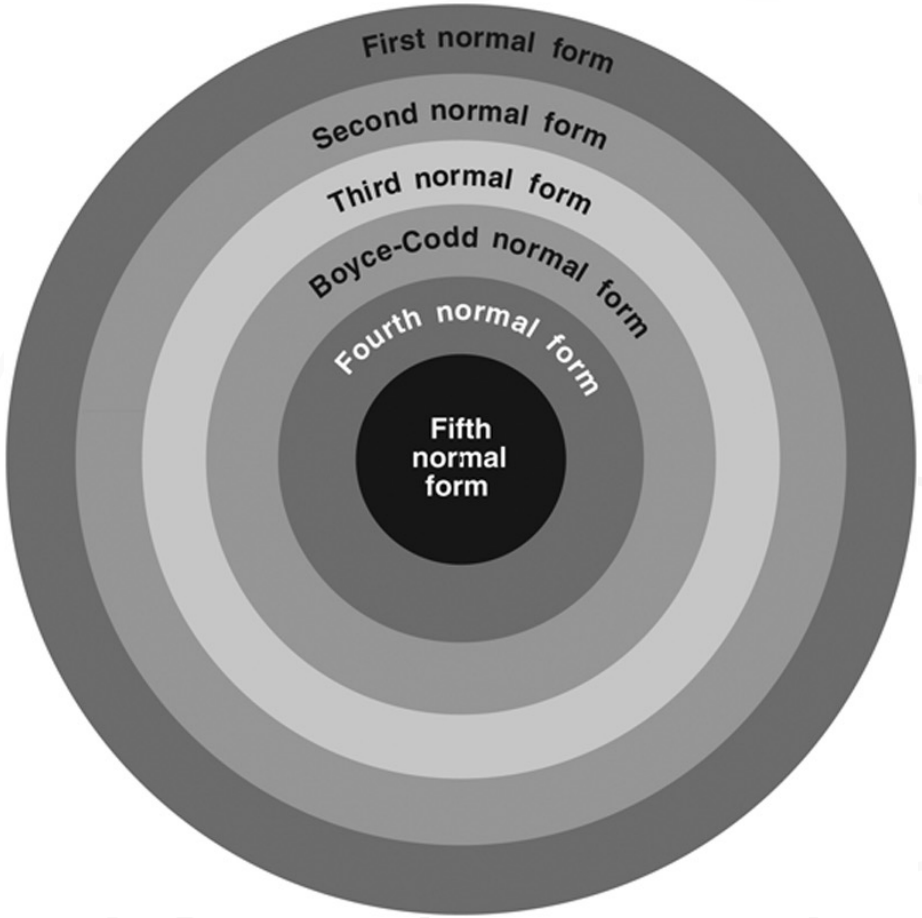
Foreign keys

Duke

Eliminating redundancy: Normalization

Normalization: "The process of designing relations to adhere to increasingly stringent sets of rules to avoid problems with poor database design."

Normal form: "A set of theoretical rules to which a relation must conform."



"Relational Database Design and Implementation, 4th Edition", Harrington
https://learning.oreilly.com/library/view/relational-database-design/9780128499023/B9780128043998000399/B9780128043998000399.xhtml#st0075

Duke

First Normal Form

- The data are stored in a two-dimensional table. (Think 'spreadsheet', but...
  - Contains only atomic values
  - No repeating groups.
- Not this:

Name	Accounts
George Blake	Checking, Savings

- And not this:

Name	Account1	Account 2	Account 3
George Blake	Checking	Savings	
Steve Smith	Checking	Savings	CD

This:

First Name	Last Name	Account
George	Blake	Checking
George	Blake	Savings

First Normal Form

- The data are stored in a two-dimensional table. (Think 'spreadsheet', but...
  - Contains only atomic values
  - No repeating groups.
- Not this:

Name	Accounts
George Blake	Checking, Savings

- And not this:

Name	Account1	Account 2	Account 3
George Blake	Checking	Savings	
Steve Smith	Checking	Savings	CD

Or this:

cust_id	fname	lname
1	George	Blake

cust_id	Account
1	Checking
1	Savings

"Relational Database Design and Implementation, 4th Edition", Harrington

Duke

Second Normal Form

- The relation is in first normal form.
- All non-key attributes are functionally dependent on the entire primary key.

"A functional dependency is a one-way relationship between two attributes, such that at any given time, for each unique value of attribute A, only one value of attribute B is associated with it throughout the relation"

cust_id	fname	lname
1	George	Blake
2	Sue	Smith

cust_id	account_id	Account
1	103	Checking
1	104	Savings
2	105	Checking
2	106	Money Market
2	107	Line of Credit

"Relational Database Design and Implementation, 4th Edition", Harrington

Third Normal Form

- The relation is in second normal form.
- There are no transitive dependencies.

Not:

account_id	product_code	product_name	cust_id	balance
103	CHK	Checking	1	$75.00
104	SAV	Savings	1	$250.00

This:

Customer

cust_id	fname	lname
1	George	Blake
2	Sue	Smith

Account

account_id	product_cd	cust_id	balance
103	CHK	1	$75.00
104	SAV	1	$250.00
105	CHK	2	$783.64
106	MM	2	$500.00
107	LOC	2	0

Product

product_cd	name
CHK	Checking
SAV	Savings
MM	Money market
LOC	Line of credit

Duke

Relational Model

Customer

cust_id	*fname*	*lname*
1	George	Blake
2	Sue	Smith

Product

product_cd	*name*
CHK	Checking
SAV	Savings
MM	Money market
LOC	Line of credit

Account

account_id	*product_cd*	*cust_id*	*balance*
103	CHK	1	$75.00
104	SAV	1	$250.00
105	CHK	2	$783.64
106	MM	2	$500.00
107	LOC	2	0

Transaction

txn_id	*txn_type_cd*	*account_id*	*amount*	*date*
978	DBT	103	$100.00	2004-01-22
979	CDT	103	$25.00	2004-02-05
980	DBT	104	$250.00	2004-03-09
981	DBT	105	$1000.00	2004-03-25
982	CDT	105	$138.50	2004-04-02
983	CDT	105	$77.86	2004-04-04
984	DBT	106	$500.00	2004-03-27

George Blake withdrew $100 from a checking account on 1/22/2004

Duke

From 'Learning SQL', Beaulieu, https://learning.oreilly.com/library/view/learning-sql-3rd/9781492057604/

SQL: Data Definition Language (DDL)

Account

account_id	product_cd	cust_id	balance
103	CHK	1	$75.00
104	SAV	1	$250.00
105	CHK	2	$783.64
106	MM	2	$500.00
107	LOC	2	0

```
DDL:
CREATE TABLE Account (
  account_id   INT      NOT NULL,
  product_cd   CHAR(3)  NOT NULL,
  cust_id      INT      NOT NULL,
  balance      NUMERIC  NOT NULL,
  PRIMARY KEY (account_id)
);
```

Specification for creating the database schema

Every DBMS maintains a data dictionary - metadata

Duke

Database Modeling

Entity-Relationships

Duke

Entity Relationship Modeling

Office

Course

Faculty

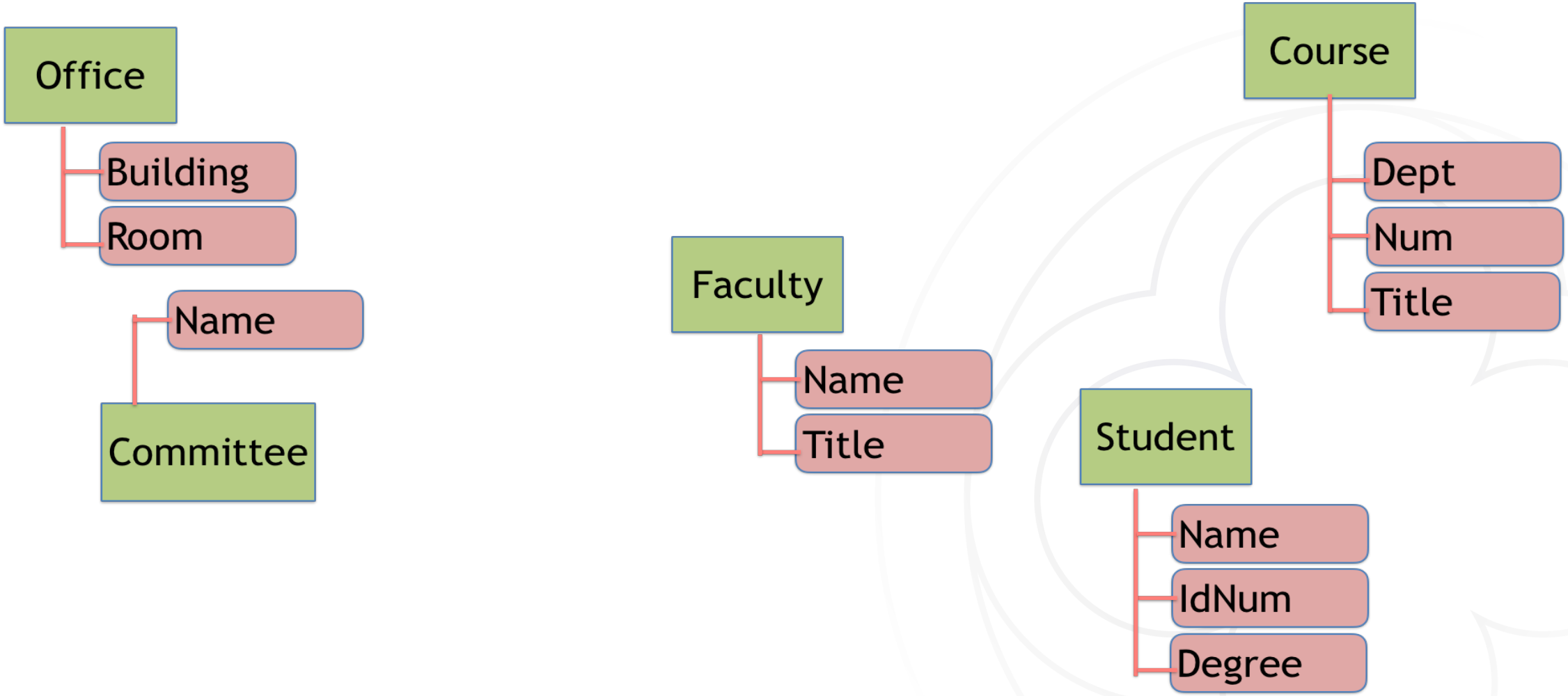
Committee

Student

Entities (Nouns)

Duke

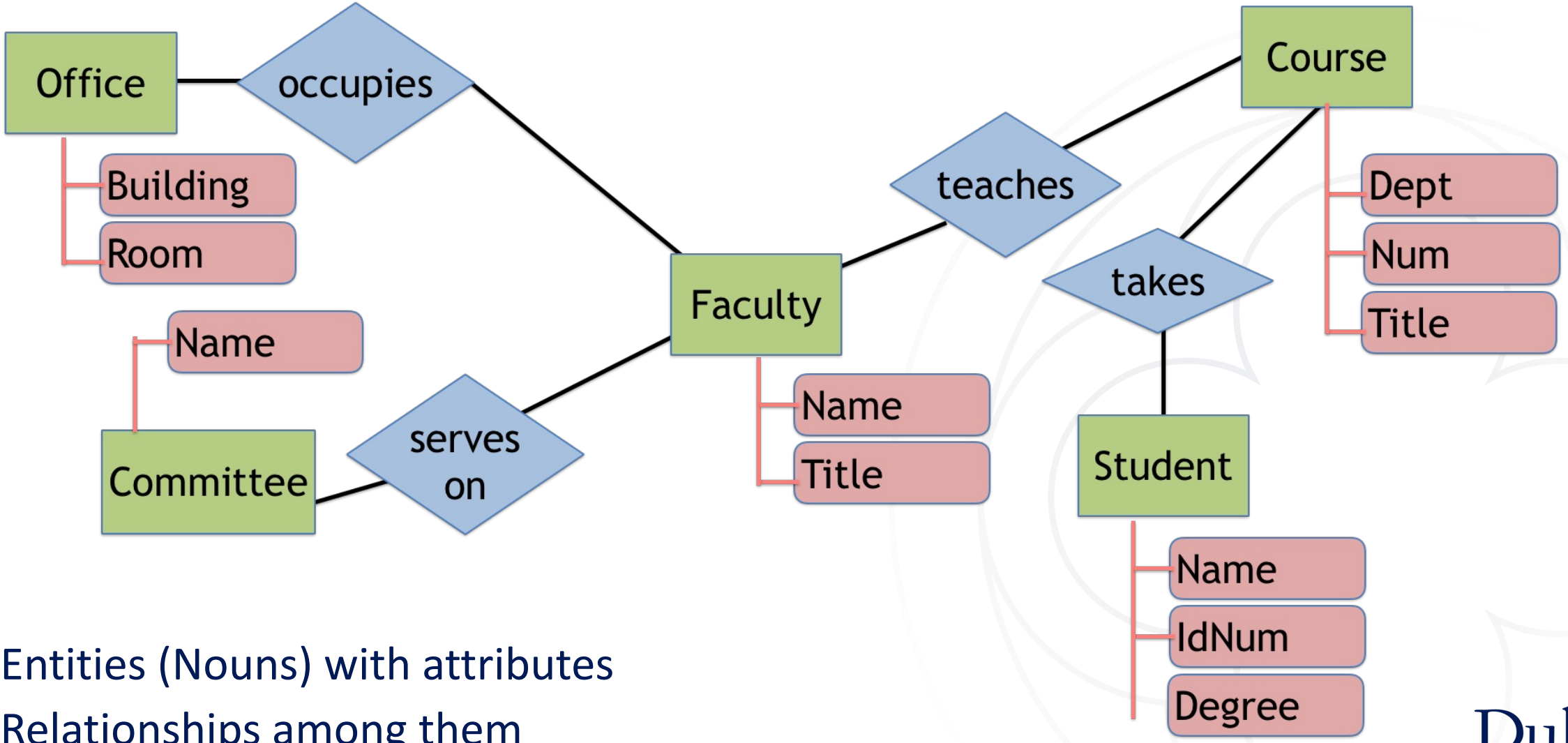
Entity Relationship Modeling



Entities (Nouns) with **attributes**

Duke

Entity Relationship Modeling

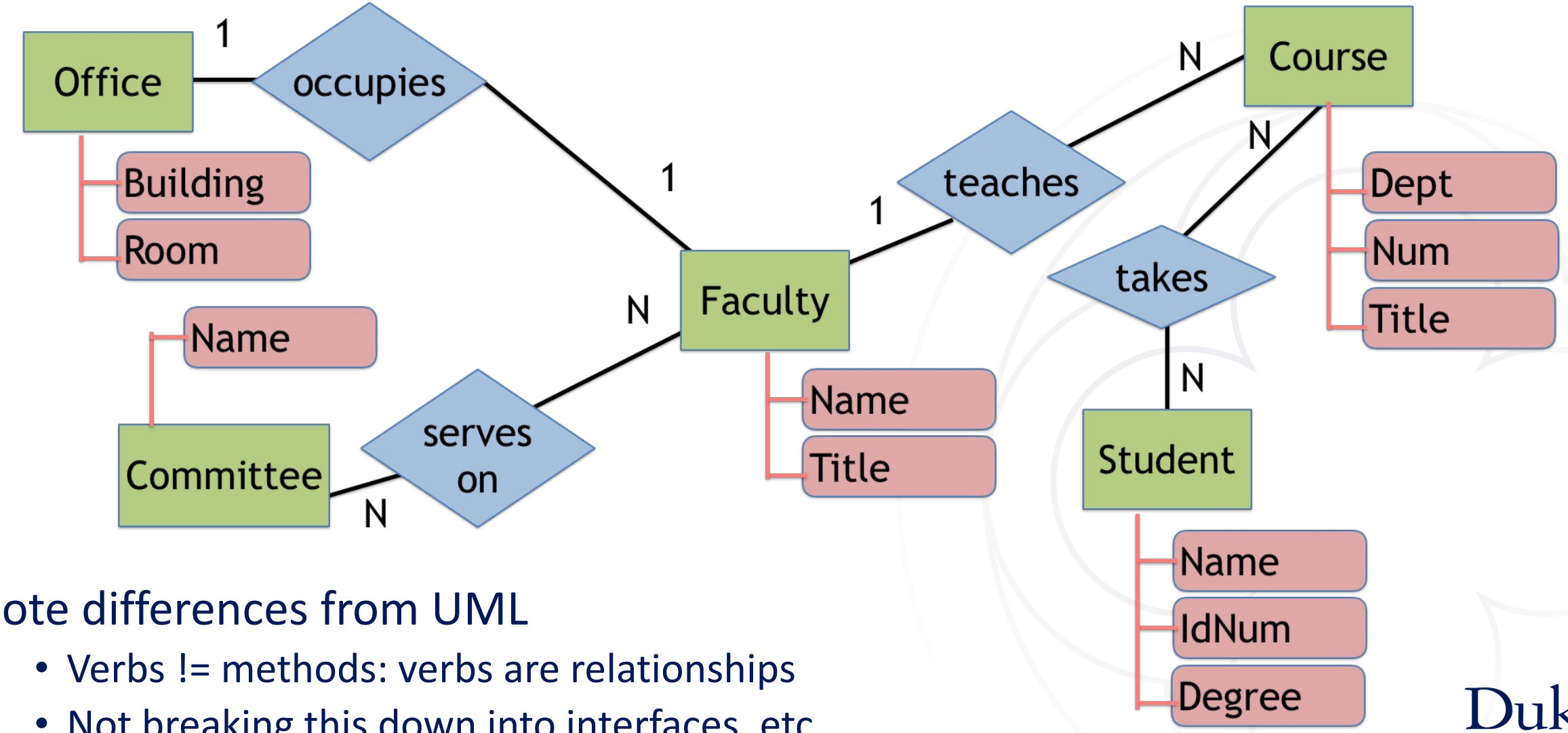


Entities (Nouns) with attributes

Relationships among them

Duke

Entity Relationship Modeling



Note differences from UML

- Verbs != methods: verbs are relationships
- Not breaking this down into interfaces, etc..

Duke

ER Model Summary

- Identify business objects
- Represent those objects as entities
- Determine relationships among those entities
- Create ER Diagram

Now, we need to create tables …

But first, revisit normalization …

- We normalize our database design to prevent problems:
  - Deleting data results in the inadvertent deletion of other data
  - Updating table data takes too long or is done incompletely
  - Tables contain redundant data
  - Searching for specific data takes too long

Maximize the efficiency of a database.

Duke

Normalization Guidelines

- First Normal Form
  - For each relation:
    - Each attribute has an atomic value
    - There are no repeating groups
    - There are no duplicate rows
  - Benefit: any data item can be uniquely identified by relation, attribute, and key.
- Second Normal Form
  - The relation is in first normal form
  - All non-key attributes are functionally dependent on the entire primary key
- Third Normal Form
  - The relation is in second normal form
  - There are no transitive dependencies

Duke

Is this Relation Normalized?

FacultyID	Name	Title	TeachesCourse	ClassroomID	Committees
pjm52	Pat Morrison	Asst. Professor	FINTECH510	HUDSON 207	FinTech Development, Recruiting
pjm52	Pat Morrison	Asst. Professor	FINTECH512	TEER 106	FinTech Development, Recruiting
jbs108	John Slankas	Lecturer	FINTECH512	TEER 106	

Duke

First Normal Form

- All values are atomic
- No repeating values

FacultyID	LastName	FirstName	Title	TeachesCourse	ClassroomID
pjm52	Morrison	Pat	Asst. Professor	FINTECH510	HUDSON 207
pjm52	Morrison	Pat	Asst. Professor	FINTECH512	TEER 106
jbs108	Slankas	John	Lecturer	FINTECH512	TEER 106

FacultyID	Committee
pjm52	FinTech Development
pjm52	Recruiting

Now normalized?

Duke

Second Normal Form

All non-key attributes are functionally dependent on the entire primary key

FacultyID	LastName	FirstName	Title	TeachesCourse
pjm52	Morrison	Pat	Asst. Professor	FINTECH510
pjm52	Morrison	Pat	Asst. Professor	FINTECH512
jbs108	Slankas	John	Lecturer	FINTECH512

Now normalized?

FacultyID	Committee
pjm52	FinTech Development
pjm52	Recruiting

CourseID	ClassroomID
FINTECH510	HUDSON 207
FINTECH512	TEER 106

Duke

Normalized

FacultyID	LastName	FirstName	Title
pjm52	Morrison	Pat	Asst. Professor
jbs108	Slankas	John	Lecturer

FacultyID	TeachesCourse
pjm52	FINTECH510
pjm52	FINTECH512
jbs108	FINTECH512

FacultyID	Committee
pjm52	FinTech Development
pjm52	Recruiting

CourseID	ClassroomID
FINTECH510	HUDSON 207
FINTECH512	TEER 106

Duke

Normalization “Bumper Sticker”

Every attribute in a relation should depend only on the key, the whole key, and nothing but the key

Denormalization

It’s ok for a database to not be completely normalized, but this should be a conscious decision

- Improve database performance
- Reduce unnecessary complexity

In Class Exercise

Transactions

Transaction

Unit of program execution that accesses and possibly updates various data items.

- Transfer $50 from account A to account B:
  1. **read**(A)
  2. $A := A - 50$
  3. **write**(A)
  4. **read**(B)
  5. $B := B + 50$
  6. **write**(B)
- Two main issues to deal with:
  - Failures of various kinds, such as hardware failures and system crashes
  - Concurrent execution of multiple transactions

Duke

Why Concurrency Control is needed

- **The Lost Update Problem**
  - Occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database item incorrect.
- **The Temporary Update (or Dirty Read) Problem**
  - Occurs when one transaction updates a database item and then the transaction fails for some reason
  - Updated item is accessed by another transaction before it is changed back to its original value.
- **The Incorrect Summary Problem**
  - If one transaction is calculating an aggregate summary function on a number of records while other transactions are updating some of these records, the aggregate function may calculate some values before they are updated and others after they are updated.

Duke

Transaction Processing

- **Single-User System**:
  - At most one user at a time can use the system.
- **Multiuser System**:
  - Many users can access the system concurrently.
- **Concurrency**
  - **Interleaved processing**:
    - Concurrent execution of processes is interleaved in a single CPU
  - **Parallel processing**:
    - Processes are concurrently executed in multiple CPUs.

Required Properties of a Transaction

Consider a transaction to transfer $50 from account A to account B:

1. **read**(*A*)
2. *A* := *A* – 50
3. **write**(*A*)
4. **read**(*B*)
5. *B* := *B* + 50
6. **write**(*B*)

Atomicity requirement

- If the transaction fails after step 3 and before step 6, money will be "lost" leading to an inconsistent database state
  - Failure could be due to software or hardware
- The system should ensure that updates of a partially executed transaction are not reflected in the database

Durability requirement — once the user has been notified that the transaction has completed (i.e., the transfer of the $50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures.

Duke

Required Properties of a Transaction

Consistency requirement

- The sum of A and B is unchanged by the execution of the transaction
- Consistency requirements include
    - Explicitly specified integrity constraints such as primary keys and foreign keys
    - Implicit integrity constraints
        - e.g., sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
- A transaction, when starting to execute, must see a consistent database.
- During transaction execution the database may be temporarily inconsistent.
- When the transaction completes successfully the database must be consistent
    - Erroneous transaction logic can lead to inconsistency

Required Properties of a Transaction

Isolation requirement –

If between steps 3 and 6 (of the fund transfer transaction) , another transaction **T2** is allowed to access the partially updated database, it will see an inconsistent database (the sum *A + B* will be less than it should be).

	T1	**T2**
1.	**read**(*A*)	
2.	*A* := *A* – 50	
3.	**write**(*A*)	
		read(A), read(B), print(A+B)
4.	**read**(*B*)	
5.	*B* := *B* + 50	
6.	**write**(*B*	

- Isolation can be ensured trivially by running transactions **serially**

- However, executing multiple transactions concurrently has significant benefits

Duke

Transaction

1. Begin transaction	
2. read(A)	acquires shared lock on A
3. A := A – 50	
4. write(A)	acquires exclusive lock on A
5. read(B)	acquires shared lock on B
6. B := B + 50	
7. write(B)	acquires exclusive lock on B
8. Commit transaction	releases all locks

NoSQL

Houston, we have too much data

Duke

Data Management: Trends & Requirements

Trends

- **Volume** of data
- **Cloud** comp. (IaaS)
- **Velocity** of data
- **Big** users
- **Variety** of data

Requirements

- Real database **scalability**
  - massive database distribution
  - dynamic resource management
  - horizontally scaling systems
- Frequent **update** operations
- Massive **read** throughput
- **Flexible** database schema
  - semi-structured data

Duke

RDBMS for Big Data

- relational schema
  - data in tuples
  - a priori known schema
- schema normalization
  - data split into tables (3NF)
  - queries merge the data
- transaction support
  - trans. management with ACID
  - Atomicity, Consistency, Isolation, Durability
  - safety first

- but current data are naturally flexible
- inefficient for large data
- slow in distributed environment
- full transactions very inefficient in distributed environment

NoSQL Databases

- What is "NoSQL"?
  - term used in late 90s for a different type of technology: Carlo Strozzi: http://www.strozzi.it/cgi-bin/CSA/tw7/I/en_US/NoSQL/
  - "Not Only SQL"?
    - but many RDBMS are also "not just SQL"

"NoSQL is an accidental term with no precise definition"

- first used at an informal meetup in 2009 in San Francisco (presentations from Voldemort, Cassandra, Dynomite, HBase, Hypertable, CouchDB, and MongoDB)

[Sadalage & Fowler: NoSQL Distilled, 2012]

Duke

NoSQL Databases (cont.)

- NoSQL: Database technologies that are (mostly):
  - Not using the relational model (nor the SQL language)
  - Designed to run on large clusters (horizontally scalable)
  - No schema - fields can be freely added to any record
  - Open source
  - Based on the needs of 21st century web estates

  [Sadalage & Fowler: NoSQL Distilled, 2012]

- Other characteristics (often true):
  - easy replication support (fault-tolerance, query efficiency)
  - simple API
  - eventually consistent (not ACID)

Duke

Review

- Database modeling
  - Entity-Relationship diagrams
  - Normalization
  - Data anomalies and their resolution
- Joins
- Transactions
- NoSQL